Guia de Estudo

Aplicação Java do Sistema do Bar

Projeto de Banco de Dados · Entrega 03 · Spring Boot + PostgreSQL

Divisão da apresentação

	Pessoa 1: Estrutura e CRUD	Pessoa 2: Consultas e Dashboard
Assuntos	Visão geral e arquitetura, conceitos de Spring, CRUD de Clientes/Mesas/Produtos, transações, tratamento de erros e configuração	As 5 consultas, visualizador de tabelas, extrator genérico de resultados, dashboard estatístico, teste t (p-valor) e gráficos
Arquivos	BarApplication, *Controller (Cliente, Mesa, Produto), ErrosBanco, ConexaoAdvice, ClienteRepository, MesaRepository, ProdutoRepository, model/*, application.properties	ConsultaController, TabelaController, DashboardController, ConsultaRepository, TabelaRepository, DashboardRepository, Resultados, TesteT, dashboard.js
Demonstração	Inserir, editar e excluir cliente; erros de chave duplicada e de FK	Abrir dashboard, uma consulta e uma tabela
Tempo sugerido	~ 5 a 6 min	~ 5 a 6 min

Como usar este guia

A **Parte Comum** (seções 1 e 2) os dois precisam saber: se o professor fizer uma pergunta geral para qualquer um, ambos devem conseguir responder.

Cada parte termina com **perguntas prováveis**. Treine respondendo em voz alta sem olhar.

Parte Comum: os dois precisam saber

1. O que é a aplicação

É uma aplicação **web** feita em **Spring Boot 3.5** com **Java 17**, que se conecta ao banco **PostgreSQL** criado nas entregas 01 e 02. Ela tem quatro áreas:

- **Dashboard**: indicadores e gráficos estatísticos do bar.
- **CRUD**: inserir, alterar, excluir e listar **Clientes**, **Mesas** e **Produtos**.
- **Consultas**: as 5 consultas SQL da entrega, mostrando o SQL ao lado do resultado.
- **Tabelas**: visualização do conteúdo de cada uma das 9 tabelas do banco.

Frase de abertura sugerida:

> *"Nossa aplicação é um sistema web em Spring Boot que usa o banco PostgreSQL que modelamos. Todo o SQL foi escrito à mão com JdbcTemplate, sem ORM, para que as consultas, os JOINs e as transações ficassem explícitos."*

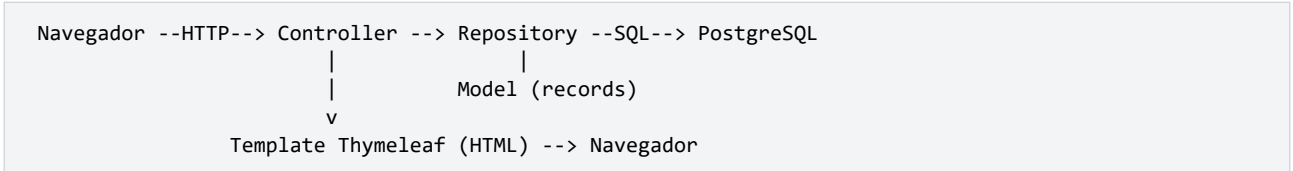
Tecnologias (pom.xml)

Dependência	Para que serve
spring-boot-starter-web	Servidor web embutido (Tomcat) e os controllers que respondem às URLs
spring-boot-starter-thymeleaf	Motor de templates: gera o HTML no servidor com os dados do banco
spring-boot-starter-jdbc	JdbcTemplate (executa SQL) e o pool de conexões HikariCP
postgresql	Driver JDBC do PostgreSQL
Chart.js (no HTML)	Biblioteca JavaScript que desenha os gráficos do dashboard

Por que JDBC e não JPA/Hibernate?

Porque é uma disciplina de banco de dados. Com JPA o SQL é gerado automaticamente e fica escondido. Com **JdbcTemplate** cada SELECT, INSERT, UPDATE e DELETE está escrito no código, então dá para mostrar JOIN, CTE, funções de janela e funções de agregação.

2. Arquitetura em camadas (MVC)



Pacote	Responsabilidade
controller	Recebe a requisição HTTP, chama o repositório, coloca os dados no Model e escolhe a página
repository	Somente acesso ao banco: cada método executa um SQL
model	Records com os dados que circulam entre as camadas (Cliente, Mesa, Produto...)
estatistica	TesteT: cálculo do p-valor que o PostgreSQL não faz
resources/templates	Páginas HTML (Thymeleaf)

O **BarApplication** só tem o `main`. A anotação `@SpringBootApplication` faz o Spring procurar as classes com `@Controller` e `@Repository`, criar uma instância de cada e ligá-las entre si.

Roteiro da apresentação (ordem)

#	Quem	O quê
1	Pessoa 1	Abertura: o que é a aplicação, tecnologias e arquitetura
2	Pessoa 1	Conceitos de Spring + CRUD (código do ClienteRepository) + demo do CRUD e dos erros
3	Pessoa 1 → 2	Transição: “Além do CRUD, a aplicação também consulta e analisa os dados...”
4	Pessoa 2	Consultas e tabelas (extrator genérico) + demo de uma consulta
5	Pessoa 2	Dashboard: estatística no SQL, p-valor no Java, gráficos + demo
6	Pessoa 2	Fechamento

Pessoa 1: Estrutura, CRUD e Tratamento de Erros

3. Conceitos de Java/Spring usados no projeto

Injeção de dependência pelo construtor

Ninguém escreve `new ClienteRepository()`. O Spring cria o objeto e entrega pelo construtor. Os repositórios recebem o `JdbcTemplate` do mesmo jeito, já configurado com os dados do application.properties.

```
@Controller
public class ClienteController {
    private final ClienteRepository clientes;

    public ClienteController(ClienteRepository clientes) {   // o Spring injeta
        this.clientes = clientes;
    }
```

Records (pacote model)

São classes de dados **imutáveis** (Java 16+). O Java gera automaticamente o construtor, os getters (`c.nome()`), o equals, o hashCode e o toString.

```
public record Mesa(Integer numeroMesa, Integer capacidade, String localizacao) { }
```

RowMapper: de linha do banco para objeto

É uma lambda que o JdbcTemplate chama para **cada linha** do ResultSet:

```
private static final RowMapper<Mesa> MAPPER = (rs, i) -> new Mesa(
        rs.getInt("Numero_Mesa"),
        rs.getInt("Capacidade"),
        rs.getString("Localizacao"));

public List<Mesa> listar() {
    return jdbc.query("SELECT Numero_Mesa, Capacidade, Localizacao FROM MESA ORDER BY Numero_Mesa", MAPPER);
}
```

Optional

`buscar()` retorna `Optional<Mesa>` porque o registro pode não existir. Isso obriga quem chama a tratar o caso vazio (`.orElse(null)`).

Parâmetros ? e SQL injection

Todo valor vindo do usuário vai como parâmetro `?`. O JdbcTemplate usa **PreparedStatement**: o SQL e os valores vão separados para o banco, então um texto malicioso nunca é executado como SQL.

```
jdbc.update("UPDATE MESA SET Capacidade = ?, Localizacao = ? WHERE Numero_Mesa = ?",
        m.capacidade(), m.localizacao(), m.numeroMesa());
```

4. Fluxo do CRUD (igual para Cliente, Mesa e Produto)

Ação	URL	Método do controller	O que faz
Listar	GET /clientes	lista()	Põe no Model a lista e um formulário vazio
Abrir edição	GET /clientes?editar=CPF	lista()	Formulário vem preenchido e edicao = true
Salvar	POST /clientes/salvar	salvar()	Se edicao: atualizar(); senão: inserir()
Excluir	POST /clientes/{cpf}/excluir	excluir()	Chama excluir() (o HTML pede confirm())

Post/Redirect/Get + Flash Attributes

Depois de um POST, o controller **não** devolve a página diretamente. Ele retorna `redirect:/clientes` e a mensagem de sucesso ou erro vai em `addFlashAttribute`, que sobrevive a um redirect. Assim, se o usuário der F5, o formulário não é reenviado e o registro não é inserido duas vezes.

```
try {
    if (edicao) { clientes.atualizar(c); ra.addFlashAttribute("sucesso", "Cliente ... alterado."); }
    else        { clientes.inserir(c);   ra.addFlashAttribute("sucesso", "Cliente ... inserido."); }
} catch (DataAccessException e) {
    ra.addFlashAttribute("erro", ErrosBanco.mensagem(e));
    return edicao ? "redirect:/clientes?editar=" + c.cpf() : "redirect:/clientes";
}
return "redirect:/clientes";
```

Antes de salvar, o controller **limpa os dados**: `cpf.replaceAll("\\D", "")` remove pontos e traços, `trim()` tira espaços e `vazioParaNulo()` grava NULL em vez de texto vazio nos campos opcionais (rua, número, bairro, CEP).

Em Mesa e Produto, o formulário de inserção já sugere o próximo código: `SELECT COALESCE(MAX(Numero_Mesa), 0) + 1 FROM MESA`.

5. ClienteRepository: o ponto alto (modelagem refletida no código)

> **Por que Cliente é diferente de Mesa e Produto**
>
> **Especialização:** um cliente é uma linha em **PESSOA** + uma linha em **CLIENTE** com o mesmo CPF.
>
> **Atributo multivalorado:** os telefones ficam na tabela **TELEFONE** (CPF, Telefone).
>
> Por isso uma operação de cliente mexe em **3 tabelas** e precisa ser uma **transação**.

Listagem: JOIN + LEFT JOIN + STRING_AGG

```
SELECT p.CPF, p.Nome, p.Rua, p.Numero, p.Bairro, p.CEP,
       c.Data_Primeira_Visita, c.Fidelidade_Ativa,
       COALESCE(STRING_AGG(t.Telefone, ', ' ORDER BY t.Telefone), '') AS Telefones
  FROM CLIENTE c
  JOIN PESSOA p ON p.CPF = c.CPF
  LEFT JOIN TELEFONE t ON t.CPF = p.CPF
 GROUP BY p.CPF, p.Nome, ... , c.Fidelidade_Ativa
```

- **JOIN** PESSOA: junta os dados gerais com os dados de cliente.
- **LEFT JOIN** TELEFONE: o cliente *sem* telefone também aparece.
- **STRING_AGG**: junta os vários telefones em uma única string "8199..., 8198...".
- **COALESCE**: se não houver telefone, mostra '' em vez de NULL.

Inserir / atualizar / excluir com @Transactional

```
@Transactional
public void inserir(Cliente c) {
    jdbc.update("INSERT INTO PESSOA (CPF, Nome, Rua, Numero, Bairro, CEP) VALUES (?, ?, ?, ?, ?, ?)", ...);
    jdbc.update("INSERT INTO CLIENTE (CPF, Data_Primeira_Visita, Fidelidade_Ativa) VALUES (?, ?, ?)", ...);
    inserirTelefones(c.cpf(), c.telefones());
}

@Transactional
public void atualizar(Cliente c) {
    jdbc.update("UPDATE PESSOA SET ... WHERE CPF = ?", ...);
    jdbc.update("UPDATE CLIENTE SET ... WHERE CPF = ?", ...);
    jdbc.update("DELETE FROM TELEFONE WHERE CPF = ?", c.cpf());   // substitui a lista inteira
    inserirTelefones(c.cpf(), c.telefones());
}

@Transactional
public void excluir(String cpf) {
    jdbc.update("DELETE FROM CLIENTE WHERE CPF = ?", cpf);
    jdbc.update("DELETE FROM PESSOA WHERE CPF = ?", cpf);        // TELEFONE sai por ON DELETE CASCADE
}
```

- **@Transactional** = **atomicidade** (o A do ACID): ou todos os comandos são gravados, ou nenhum. Se o INSERT em CLIENTE falhar (ex.: CPF duplicado), o INSERT em PESSOA é desfeito (rollback).
- Na atualização, o telefone é multivalorado: o mais simples e correto é **apagar todos e inserir a lista nova**.
- Na exclusão, se o cliente já tem **ATENDIMENTO**, a FK bloqueia o DELETE, a transação inteira volta e nada é apagado.
- `inserirTelefones` separa a string por vírgula, ponto e vírgula ou espaço, remove o que não é dígito, ignora vazios e duplicados (`distinct()`) e faz um INSERT por telefone.

6. Tratamento de erros

ErrosBanco: traduz o SQLSTATE do PostgreSQL

Quando o banco rejeita uma operação, o Spring lança `DataAccessException`. O controller captura a exceção e o **ErrosBanco** lê o código SQLSTATE para mostrar uma mensagem em português:

SQLSTATE	Significado	Exemplo no sistema
23505	unique_violation: chave duplicada	Inserir cliente com CPF já existente
23503	foreign_key_violation	Excluir produto que já aparece em PEDIDO
23514	check_violation	Preço/capacidade/salário <= 0 (o HTML já barra antes, o banco é a 2ª barreira)
23502	not_null_violation	Campo obrigatório vazio
22001	string_data_right_truncation	Texto maior que o VARCHAR da coluna

> **Frase forte para a apresentação**
>
> As **regras de integridade ficam no banco** (PK, FK, CHECK, NOT NULL). O Java não duplica essas validações, só mostra a mensagem. Assim o banco garante a consistência mesmo que alguém acesse por outro programa.

ConexaoAdvice

Um `@ControllerAdvice` com `@ExceptionHandler(CannotGetJdbcConnectionException.class)`. Se o PostgreSQL estiver desligado, qualquer página mostra *erro-conexao.html* com instruções, em vez de uma stack trace.

7. Configuração (application.properties)

```
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5432/bar}
spring.datasource.username=${DB_USER:postgres}
spring.datasource.password=${DB_PASSWORD:bd2026}
spring.sql.init.mode=never                              # schema vem dos scripts SQL, não do Java
spring.datasource.hikari.initialization-fail-timeout=-1 # sobe mesmo sem banco no ar
```

- `${DB_URL:padrão}`: usa a variável de ambiente se existir, senão o valor padrão.
- **init.mode=never**: a aplicação não cria tabelas. O schema e os dados vêm dos arquivos de sql/ (entregas 01 e 02).
- **HikariCP** é o pool de conexões: reaproveita conexões abertas em vez de abrir uma por requisição.

8. Demonstração da Pessoa 1

- Abrir **Clientes** e inserir um cliente novo com 2 telefones separados por vírgula.
- Clicar em **editar**, mudar um telefone e salvar. Mostrar que a lista mudou.
- Tentar inserir o **mesmo CPF** de novo para mostrar o erro 23505.
- Tentar excluir um cliente que **tem atendimentos** para mostrar o erro de FK 23503 (nada foi apagado).
- Excluir o cliente recém-criado: funciona, e os telefones somem junto (CASCADE).
- (Opcional) Em **Produtos**: o formulário já barra preço menor que 0,01 (atributo min do HTML). Explique que o **CHECK (Preco_Atual > 0)** do banco é a segunda barreira, que vale mesmo fora da aplicação.

9. Perguntas prováveis da Pessoa 1

Pergunta	Resposta
Onde está o SQL?	Nos *Repository, como strings (text blocks """), executado pelo JdbcTemplate.
Como vocês evitam SQL injection?	Todos os valores vão como parâmetros ? (PreparedStatement).
Para que serve o @Transactional?	Garante atomicidade: INSERT/UPDATE/DELETE em PESSOA, CLIENTE e TELEFONE juntos ou nada.
O que acontece se o INSERT em CLIENTE falhar?	Exceção, rollback automático, e o INSERT em PESSOA também é desfeito.
Por que DELETE em CLIENTE antes de PESSOA?	CLIENTE.CPF referencia PESSOA.CPF (FK). Apagar a pessoa antes violaria a FK.
Por que usar records?	Dados imutáveis e menos código (construtor/getters automáticos).
Por que redirect depois do POST?	Padrão Post/Redirect/Get: evita reenviar o formulário no F5.
E se dois usuários pegarem o mesmo MAX+1?	A PK barra o segundo e ele recebe a mensagem de chave duplicada. Uma SEQUENCE/SERIAL resolveria.
Como o Spring sabe qual método chamar?	Pelas anotações @GetMapping/@PostMapping com a URL; @RequestParam e @PathVariable leem os valores.

Pessoa 2: Consultas, Tabelas e Dashboard

10. Consultas (ConsultaController + ConsultaRepository)

As 5 consultas ficam numa lista fixa de objetos `Consulta` (record com id, título, descrição, recursos SQL e o SQL). O mesmo SQL está em *sql/consultas/consultas.sql*.

```
public record Consulta(String id, String titulo, String descricao, List<String> recursos, String sql) { }

@GetMapping("/consultas/{id}")
public String executar(@PathVariable String id, Model model) {
    Consulta consulta = consultas.buscar(id)
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));   // id inválido = 404
    model.addAttribute("resultado", consultas.executar(consulta));
    ...
}
```

`GET /consultas` só redireciona para a primeira consulta. A página mostra o SQL ao lado do resultado, para o professor ver exatamente o que rodou.

As 5 consultas (saiba uma frase de cada)

Consulta	O que responde	Recursos SQL
1. Faturamento por categoria	Quanto cada categoria faturou, itens vendidos e % do total	JOIN, GROUP BY, SUM, COUNT DISTINCT, **SUM(SUM(...)) OVER ()** para o total geral
2. Ranking de atendentes	Garçons ordenados pelo faturamento, ticket médio e supervisor	JOIN de 6 tabelas, **LEFT JOIN** PESSOA do supervisor, **RANK() OVER**
3. Conferência de contas	Compara consumido x pago em cada atendimento	**2 CTEs** (consumo, pagamentos), LEFT JOIN, COALESCE, **CASE** (QUITADA / PENDENTE / PAGO A MAIS)
4. Clientes acima da média	Clientes que gastam mais que a média, com o produto favorito	CTE, **subconsulta com AVG no WHERE**, **subconsulta correlacionada** com LIMIT 1
5. Hierarquia de supervisão	Cada gerente, quantos subordinados, cargos e folha da equipe	**Autojunção** FUNCIONARIO s JOIN FUNCIONARIO f ON f.CPF_Supervisor = s.CPF, STRING_AGG, SUM/AVG

> **Detalhe que pode ser perguntado**
>
> O valor de cada item é `Quantidade * Preco_Atual`, porque a tabela PEDIDO não guarda o preço cobrado no momento da venda.

11. Resultados.EXTRATOR: um leitor genérico de SELECT

Cada consulta retorna colunas diferentes, então não dá para ter um record para cada uma. A solução foi um `ResultSetExtractor` que funciona para **qualquer** SELECT, lendo o **ResultSetMetaData** (os metadados das colunas):

```
static final ResultSetExtractor<ResultadoTabela> EXTRATOR = rs -> {
    ResultSetMetaData meta = rs.getMetaData();
    int qtd = meta.getColumnCount();
    for (int i = 1; i <= qtd; i++) {
        colunas.add(meta.getColumnLabel(i));                              // nome / alias da coluna
        numericas.add(TIPOS_NUMERICOS.contains(meta.getColumnType(i)));  // alinhar à direita?
    }
    while (rs.next()) {                                                   // percorre as linhas
        List<Object> linha = new ArrayList<>();
        for (int i = 1; i <= qtd; i++) linha.add(formatar(rs.getObject(i)));
        linhas.add(linha);
    }
    return new ResultadoTabela(colunas, numericas, linhas);
};
```

- `formatar()` mostra Timestamp como dd/MM/yyyy HH:mm, Date como dd/MM/yyyy e boolean como Sim/Nao. Números continuam números.
- O resultado é um `ResultadoTabela(colunas, numericas, linhas)`, que um único fragmento HTML desenha. **Consultas e Tabelas usam a mesma página de resultado.**

12. Visualizador de tabelas (TabelaRepository)

Mostra o conteúdo das 9 tabelas: PESSOA, TELEFONE, CLIENTE, FUNCIONARIO, MESA, PRODUTO, ATENDIMENTO, PEDIDO e PAGAMENTO_PARCIAL.

```
private static final Map<String, String> SELECTS = new LinkedHashMap<>();
static {
    SELECTS.put("PESSOA", "SELECT CPF, Nome, Rua, Numero, Bairro, CEP FROM PESSOA ORDER BY CPF");
    SELECTS.put("MESA",   "SELECT Numero_Mesa, Capacidade, Localizacao FROM MESA ORDER BY Numero_Mesa");
    ...
}
public Optional<ResultadoTabela> conteudo(String tabela) {
    String sql = SELECTS.get(tabela.toUpperCase());
    return sql == null ? Optional.empty() : Optional.of(jdbc.query(sql, Resultados.EXTRATOR));
}
```

Segurança: ponto importante

O nome da tabela vem da URL (`/tabelas/{nome}`), mas **nunca é concatenado no SQL**. Ele só serve de chave num mapa de SELECTs fixos, uma **whitelist**. Um nome que não está no mapa retorna Optional vazio e a página dá 404. Isso impede SQL injection pela URL.

LinkedHashMap mantém a ordem de inserção, então o menu mostra as tabelas na ordem do modelo.

13. Dashboard (DashboardController + DashboardRepository)

O controller chama vários métodos do repositório e coloca cada resultado no Model. Cada método é um SELECT que retorna `Map` (uma linha) ou `List<Map>` (várias linhas), com `queryForMap` / `queryForList`.

Método	O que mostra	SQL de destaque
indicadores()	Cards: faturamento, atendimentos, abertos, ticket médio, itens, clientes, fiéis	Várias subconsultas escalares num único SELECT
faturamentoPorCategoria()	Gráfico por categoria	JOIN + GROUP BY + SUM
faturamentoPorDia()	Linha do tempo por dia	CAST(... AS DATE), TO_CHAR
itensPorHora()	Movimento por hora	EXTRACT(HOUR); ORDER BY MOD(hora+12, 24) para a noite passar da meia-noite

Método	O que mostra	SQL de destaque
pagamentosPorMetodo()	Pix, cartão, dinheiro...	GROUP BY Metodo_Pagamento
produtosMaisVendidos()	Top 10 produtos	ORDER BY ... LIMIT 10
atendimentosPorLocalizacao()	Uso por área do bar	LEFT JOIN (área sem atendimento aparece com 0)
estatisticasDescritivas()	Média, mediana, desvio, CV, mín, Q1, Q3, máx	AVG, PERCENTILE_CONT, STDDEV_SAMP
distribuicaoValorAtendimento()	Histograma + tabela de frequências	GENERATE_SERIES, LEFT JOIN, SUM(...) OVER (ORDER BY)
permanenciaValorCorrelacao()	Correlação de Pearson e regressão	CORR, REGR_SLOPE, REGR_INTERCEPT
permanenciaValorPontos()	Gráfico de dispersão	CTEs permanencia + consumo

Estatística descritiva (no PostgreSQL)

Base: total consumido por atendimento **fechado** (CTE `totais`).

```
SELECT COUNT(*) AS n,
       ROUND(AVG(total), 2)                                          AS media,
       PERCENTILE_CONT(0.5)  WITHIN GROUP (ORDER BY total)           AS mediana,
       ROUND(STDDEV_SAMP(total), 2)                                  AS desvio_padrao,
       ROUND(100 * STDDEV_SAMP(total) / NULLIF(AVG(total), 0), 1)    AS coef_variacao,
       PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY total) AS q1, ... AS q3, MIN, MAX
  FROM totais
```

- **STDDEV_SAMP**: desvio padrão *amostral* (divide por n-1).
- **NULLIF(AVG,0)**: evita divisão por zero no coeficiente de variação.
- **Histograma**: `GENERATE_SERIES` cria todas as faixas de R$ 50 e o `LEFT JOIN` faz faixas vazias aparecerem com 0. Frequência relativa com `SUM(COUNT) OVER ()` e acumulada com `SUM(COUNT) OVER (ORDER BY faixa)`. A largura (50) vem da constante Java `LARGURA_FAIXA`, passada como parâmetro ?.

Correlação de Pearson: tempo de permanência x valor gasto

- **Permanência** = do horário de abertura da conta até o último pagamento, em minutos (`EXTRACT(EPOCH FROM ...) / 60`).
- **r** = `CORR(total, minutos)`; **r²** = coeficiente de determinação.
- Reta de mínimos quadrados: `REGR_SLOPE` (inclinação) e `REGR_INTERCEPT` (intercepto).
- Estatística de teste: **$t = r \cdot \sqrt{(n-2)} / \sqrt{(1-r^2)}$**, com **$gl = n - 2$**.
- Classificação: |r| < 0,3 fraca; < 0,7 moderada; senão forte; positiva ou negativa pelo sinal.

14. TesteT: o único cálculo estatístico feito em Java

> **Por que existe**
>
> O PostgreSQL calcula r e t, mas **não tem a distribuição t de Student**. O p-valor é calculado em Java para decidir se a correlação é significativa.

```
// DashboardController
double t  = ((Number) pearson.get("t")).doubleValue();
int    gl = ((Number) pearson.get("gl")).intValue();
double p  = TesteT.pValorBilateral(t, gl);
model.addAttribute("significativo", p < 0.05);      // nível de significância de 5%
```

Hipóteses: **H0: ρ = 0** (não há correlação) x **H1: ρ ≠ 0** (teste bilateral). Se **p < 0,05**, rejeita-se H0 e a correlação é estatisticamente significativa.

Fórmula usada:

```
p = I_x(gl/2, 1/2)      com   x = gl / (gl + t²)
```

- **I_x(a, b)** é a função **beta incompleta regularizada**; o p-valor bilateral da t de Student pode ser escrito com ela.
- Ela é calculada por **fração contínua** (algoritmo do livro *Numerical Recipes*), iterando até a diferença ficar menor que 10^{-15} ou atingir 300 iterações.
- **ln Γ(x)** (log da função gama) usa a **aproximação de Lanczos** (os 6 coeficientes no código).
- Casos especiais: gl ≤ 0 ou t = NaN retorna NaN; t infinito (r = ±1) retorna p = 0.
- Classe `final` com construtor privado: é só um utilitário com método estático.

Se perguntarem "por que não usar uma biblioteca?": para não adicionar dependência (ex.: Apache Commons Math) só para uma função, e para deixar o cálculo explícito.

15. Do Java para os gráficos

```
<!-- dashboard.html -->
<script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js"></script>
<script th:inline="javascript">
    const dados = {
        porDia:  /*[[${porDia}]]*/ [],      // o Thymeleaf troca por JSON da List<Map>
        pearson: /*[[${pearson}]]*/ {}
        ...
    };
</script>
```

Com `th:inline="javascript"`, o Thymeleaf converte as listas e mapas Java em **JSON** dentro da página. Depois o *dashboard.js* cria os gráficos com `new Chart(...)`. Os aliases SQL (`rotulo`, `valor`) viram as chaves do JSON. Como estão sem aspas, o PostgreSQL devolve em minúsculas.

16. Demonstração da Pessoa 2

- Abrir o **Dashboard**: indicadores, estatística descritiva, histograma e frequências.
- Mostrar o **gráfico de dispersão** com a reta e explicar r, r², t, gl e o p-valor (significativo ou não).
- Ir em **Consultas** e abrir a **Conferência de contas** (CTE + CASE), com o SQL ao lado do resultado.
- Ir em **Tabelas** e abrir TELEFONE ou CLIENTE para mostrar que o que a Pessoa 1 inseriu está gravado.
- (Opcional) Digitar /tabelas/xyz na URL: aparece 404, provando a whitelist.

17. Perguntas prováveis da Pessoa 2

Pergunta	Resposta
Onde é calculada a estatística?	Quase tudo no PostgreSQL (AVG, PERCENTILE_CONT, STDDEV_SAMP, CORR, REGR_*). Só o p-valor é em Java.
Por que o p-valor está no Java?	O PostgreSQL não tem a função de distribuição da t de Student.
O que significa p < 0,05?	A chance de observar essa correlação se não houvesse relação é menor que 5%, então rejeitamos H0.
Diferença entre mediana e média?	A mediana é o valor do meio e não sofre influência de valores extremos; a média sofre.
Como uma página mostra consultas diferentes?	Resultados.EXTRATOR lê os nomes e tipos das colunas do ResultSetMetaData.
O nome da tabela vem da URL. Não dá SQL injection?	Não. O nome só é chave de um mapa de SELECTs fixos (whitelist); nada é concatenado.
O que é uma função de janela?	Calcula sobre um conjunto de linhas sem agrupá-las, ex.: RANK() OVER, SUM(...) OVER ().
O que é CTE?	WITH nome AS (SELECT ...): uma subconsulta nomeada que deixa o SQL mais legível.
Subconsulta correlacionada?	Subconsulta que usa uma coluna da consulta externa (g.CPF_Cliente) e roda para cada linha.
Como os dados chegam ao gráfico?	Thymeleaf (th:inline="javascript") serializa a List em JSON e o Chart.js desenha.

Revisão final: os dois

18. Mapa rápido dos arquivos

Arquivo	Uma linha	Quem
BarApplication	main + @SpringBootApplication	P1
Cliente/Mesa/ProdutoController	CRUD: lista, salvar, excluir, com PRG e flash	P1
ClienteRepository	PESSOA + CLIENTE + TELEFONE em transação	P1
Mesa/ProdutoRepository	CRUD simples com RowMapper	P1
ErrosBanco	SQLSTATE → mensagem em português	P1
ConexaoAdvice	Página amigável se o banco estiver fora	P1
model/*	Records de dados	P1
ConsultaController/Repository	As 5 consultas com SQL e resultado	P2
TabelaController/Repository	Conteúdo das 9 tabelas por whitelist	P2
Resultados	Extrator genérico via ResultSetMetaData	P2
DashboardController/Repository	Indicadores e estatística em SQL	P2
TesteT	p-valor da t de Student (beta incompleta)	P2

19. Checklist antes de apresentar

- [ ] PostgreSQL rodando e banco **bar** criado (scripts/rodar_tudo.ps1).
- [ ] Aplicação no ar (`mvnw spring-boot:run` na pasta app) e http://localhost:8080 abrindo.
- [ ] Saber qual cliente **tem atendimento** (para a demo do erro de FK).
- [ ] IDE aberta no **ClienteRepository** (P1) e no **TesteT / DashboardRepository** (P2).
- [ ] Cada pessoa explicou a própria parte em voz alta pelo menos uma vez.
- [ ] Cada pessoa leu a parte da outra (para cobrir perguntas cruzadas).

20. Frase de fechamento sugerida

> *"Em resumo: o Java organiza a aplicação em camadas e cuida da interface, mas as regras de integridade e quase todos os cálculos ficam no PostgreSQL: constraints, transações, JOINs, CTEs, funções de janela e estatística. O Java só complementa onde o banco não chega, como o p-valor."*